



UNIVERSIDADE DE FORTALEZA
VICE REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

JOÃO ELFREDO GONSALVES DA COSTA
JOÃO VICTOR VIEIRA MARCONDES
ISABELLE BERNARDINA DA SILVA
FABRICIO GARCIA DE CARVALHO
TERESA CRISTINA LUZ COSTA CAMPOS

DOCUMENTAÇÃO TÉCNICA

TattooAli - Aplicação mobile para agendamento de tatuagens

FORTALEZA
2026

DOCUMENTAÇÃO TÉCNICA

TattooAli - Aplicação mobile para agendamento de tatuagens

Este documento contém a documentação técnica do TattooAli - Aplicação mobile para agendamento de tatuagens desenvolvido na componente curricular N393 - Projeto Aplicado Multiplataforma como requisito para obtenção de nota.

Supervisor: Prof. Bruno Lopes, Me

FORTALEZA

2026

SUMÁRIO

1. INTRODUÇÃO.....	4
1.1. CONTEXTO E JUSTIFICATIVA.....	4
1.2. OBJETIVOS.....	4
1.3. ESCOPO E DELIMITAÇÃO.....	5
2. ENGENHARIA DE REQUISITOS.....	6
2.1. REQUISITOS FUNCIONAIS (RFs).....	6
2.2. REQUISITOS NÃO FUNCIONAIS (RNFs).....	6
3. PROJETO E ARQUITETURA DO SOFTWARE.....	8
3.1. ARQUITETURA GERAL.....	8
3.2. PROJETO DO BANCO DE DADOS.....	8
3.3. PROJETO DE API.....	9
4. TECNOLOGIAS E FERRAMENTAS.....	11
4.1. STACK DE TECNOLOGIAS.....	11
4.2. FERRAMENTAS DE DESENVOLVIMENTO.....	11
5. IMPLEMENTAÇÃO E RESULTADOS.....	13
5.1. TELAS DO SISTEMA.....	13
6. AMBIENTE E GUIA DE IMPLANTAÇÃO.....	14
6.1. REQUISITOS DO AMBIENTE.....	14
6.2. PROCESSO DE IMPLANTAÇÃO.....	14
6.3. ACESSO À APLICAÇÃO IMPLANTADA.....	15
7. CONCLUSÃO.....	16
7.1. TRABALHOS FUTUROS.....	16
7.2. LIÇÕES APRENDIDAS.....	16

1. INTRODUÇÃO

1.1. CONTEXTO E JUSTIFICATIVA

O mercado da arte corporal e tatuagem tem experimentado um crescimento exponencial e uma forte aceitação cultural nos últimos anos. No entanto, a jornada do cliente – desde a busca pelo estilo ideal até o agendamento da sessão – ainda opera de maneira altamente fragmentada e informal. Atualmente, o público que deseja se tatuar depende de redes sociais genéricas para visualizar portfólios e de aplicativos de mensagens instantâneas para tentar negociar orçamentos e conciliar horários.

Essa descentralização gera atritos significativos no processo de contratação: informações desencontradas, demora no tempo de resposta, dificuldade em gerenciar a agenda de sessões e a ausência de um sistema consolidado e confiável de avaliações. Para o cliente, a falta de uma plataforma dedicada torna o processo de escolha mais arriscado e trabalhoso. Para os tatuadores e estúdios, a gestão manual de contatos e agendamentos resulta em ineficiência e, frequentemente, em perda de potenciais clientes.

*Nesse cenário, o aplicativo **TattooAli** surge como uma solução mobile inovadora para digitalizar e centralizar esse ecossistema, operando sob o modelo de serviço sob demanda (semelhante a plataformas como Airbnb e Uber). O projeto se justifica pela oportunidade de preencher a lacuna tecnológica neste nicho, conectando diretamente clientes a tatuadores em um ambiente seguro e especializado.*

Ao oferecer uma interface onde o usuário pode explorar portfólios organizados, comunicar-se diretamente através de um chat integrado, realizar agendamentos de forma autônoma, acompanhar o histórico de suas sessões e registrar avaliações, o TattooAli elimina os ruídos da contratação informal. A aplicação moderniza o acesso aos serviços de tatuagem, proporcionando uma experiência de usuário fluida, transparente e profissional, consolidando-se como uma ferramenta essencial tanto para quem busca a arte quanto para quem a produz.

1.2. OBJETIVOS

O objetivo geral deste projeto é desenvolver um aplicativo mobile que atue como uma plataforma de conexão sob demanda entre clientes e tatuadores, centralizando e simplificando a descoberta de profissionais, a comunicação e o processo de agendamento de sessões de tatuagem.

Dito isso, os objetivos específicos do projeto são:

- **Implementar um módulo de exploração de portfólios:** Permitir que os clientes visualizem galerias de trabalhos, estilos e especialidades dos tatuadores cadastrados na plataforma.
- **Desenvolver um sistema de agendamento autônomo:** Possibilitar que o usuário consulte a disponibilidade do tatuador e realize a reserva de horários para suas sessões diretamente pelo aplicativo.
- **Integrar um chat de comunicação:** Criar um canal de mensagens interno para que clientes e tatuadores possam trocar referências, tirar dúvidas e alinhar orçamentos de forma direta e segura.
- **Criar um painel de gestão de agenda:** Oferecer ao usuário uma interface para visualizar, acompanhar e gerenciar o status de suas sessões futuras e o histórico de tatuagens já concluídas.
- **Implementar um sistema de avaliações:** Desenvolver um recurso de feedback onde os clientes possam avaliar os tatuadores após a conclusão do serviço, gerando confiabilidade e reputação dentro do aplicativo.
-

1.3. ESCOPO E DELIMITAÇÃO

- **Escopo:**
 - Cadastro e autenticação de usuários (clientes).
 - Visualização de perfis de tatuadores, incluindo galerias de portfólio e informações de contato.
 - Busca e filtragem de tatuadores por nome, estilo de tatuagem (ex: Realismo, Old School, Minimalista) ou localização.

- *Sistema de chat interno para comunicação direta entre o cliente e o tatuador.*
- *Funcionalidade de solicitação e agendamento de horários para sessões.*
- *Gerenciamento da agenda do usuário, permitindo visualizar sessões futuras e o histórico de tatuagens realizadas.*
- *Sistema de avaliação e comentários (reviews) para os tatuadores após a conclusão das sessões.*
- *Integração com o sistema web*
- ***Delimitação (Fora do Escopo):***
 - *O sistema mobile não incluirá um módulo de gestão financeira ou de fluxo de caixa para os tatuadores e estúdios.*
 - *Não fará o controle de estoque de materiais e insumos (tintas, agulhas, luvas, etc.) dos profissionais.*
 - *Não haverá um módulo de e-commerce para a venda de produtos físicos (como pomadas cicatrizantes, roupas ou acessórios do estúdio).*
 - *O aplicativo não realizará, nesta versão, o processamento de pagamentos ou retenção de valores das sessões (gateway de pagamento integrado).*

2. ENGENHARIA DE REQUISITOS

2.1. REQUISITOS FUNCIONAIS (RFs)

ID	Nome do Requisito	Descrição
RF01	Autenticação e Gestão de Perfil	O aplicativo deve permitir que o cliente crie uma conta (e-mail/senha), faça login, edite suas informações pessoais e mantenha a sessão ativa no dispositivo.
RF02	Recuperação de Senha	O usuário deve poder buscar profissionais por nome (ex: Iago) ou estúdio, além de filtrar a lista por estilo de tatuagem (Realismo, Old School, Fineline, etc.) e localização.
RF03	Busca e Filtragem de Tatuadores	O usuário deve poder buscar lajes por nome e filtrar a lista por status ("Disponível", "Reservado", etc.) para encontrar rapidamente o que precisa.
RF04	Visualização de Portfólio	Ao selecionar um tatuador na lista, o aplicativo deve exibir o perfil detalhado do profissional, contendo sua biografia, especialidades e uma galeria de imagens em alta resolução de seus trabalhos.
RF05	Chat e Envio de Referências	O aplicativo deve possuir um chat interno para comunicação direta com o tatuador. O cliente deve poder utilizar a câmera ou a galeria do smartphone para capturar e enviar imagens de referência para o orçamento.
RF06	Agendamento de Sessões	O cliente deve poder acessar o calendário de disponibilidade do tatuador, selecionar uma data e horário livres, e enviar uma solicitação formal de agendamento pelo aplicativo.
RF07	Sistema de Avaliação	Após a data de conclusão de uma sessão de tatuagem, o aplicativo deve liberar uma funcionalidade para o cliente atribuir uma nota (em estrelas) e deixar um comentário público sobre o tatuador.
RF08	Acompanhamento de Agenda	O usuário deve ter acesso a uma tela "Minhas Sessões" para visualizar o status das suas solicitações (pendente, confirmado, cancelado) e consultar o histórico de tatuagens já realizadas.

2.2. REQUISITOS NÃO FUNCIONAIS (RNFs)

Desempenho:

- **RNF01:** A busca de tatuadores deve implementar um *debounce* (atraso intencional, ex: 450ms) na digitação para reduzir o número de requisições desnecessárias à API e garantir fluidez na interface.
- **RNF02:** Listas longas (como o *feed* de tatuadores e a agenda de sessões) devem utilizar renderização otimizada (**FlatList** com controle de itens em lote) e exibir *Skeleton Loading* (animações de carregamento) para melhorar a percepção de velocidade do usuário.

Usabilidade e Experiência do Usuário (UX):

- **RNF03:** O aplicativo deve garantir que o teclado virtual do dispositivo móvel não sobreponha os campos de texto em formulários (utilizando **KeyboardAvoidingView** e **ScrollViews** responsivas).
- **RNF04:** O aplicativo deve fornecer *feedback* visual claro (como *ActivityIndicator* ou *Toasts*) durante todas as operações de entrada/saída (I/O), como login, cadastro e envio de avaliações.

Compatibilidade:

- **RNF05:** O aplicativo deve ser multiplataforma, operando nativamente em sistemas Android e iOS a partir de uma única base de código em React Native.
- **RNF06:** A interface gráfica deve respeitar as *Safe Area Views* dos dispositivos, adaptando-se corretamente a *notches* (recortes de câmera) e barras de navegação do sistema operacional.

Segurança e Privacidade:

- **RNF07:** A autenticação deve ser baseada em *tokens* JWT (JSON Web Tokens) gerados pelo Supabase Auth. No cliente (mobile), o *token* deve ser armazenado de forma persistente utilizando o **AsyncStorage** local do dispositivo.

- **RNF08:** O aplicativo deve ocultar, por padrão, o preenchimento de senhas e garantir que dados sensíveis não sejam logados de forma insegura no console do dispositivo.

Confiabilidade e Resiliência:

- **RNF09:** O sistema de *chat* deve manter uma conexão em tempo real. Caso haja falha na persistência via *sockets*, o aplicativo deve implementar um sistema de *polling* de *fallback* (ex: a cada 5 segundos) para garantir a sincronização de novas mensagens sem quebrar a tela do usuário.
- **RNF10:** Em caso de falha de conexão com a API, o aplicativo deve silenciar erros de rede não-críticos em processos de retaguarda (*background*) e exibir estados amigáveis com opções de "Tentar Novamente" para o usuário.

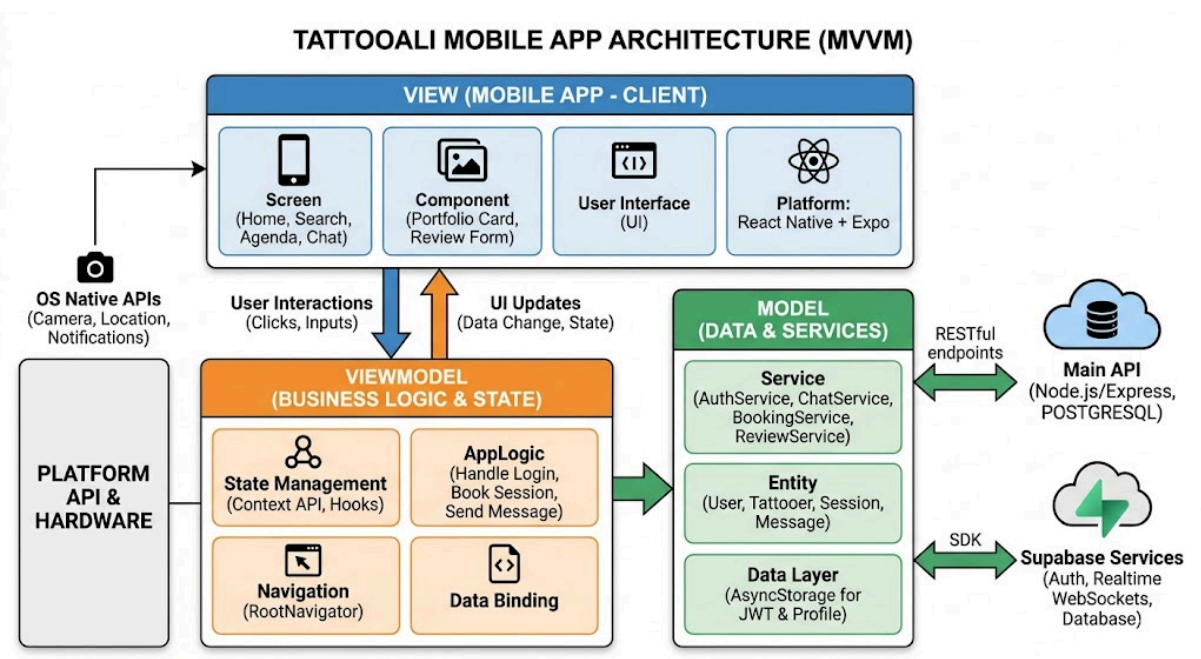
3. PROJETO E ARQUITETURA DO SOFTWARE

3.1. ARQUITETURA GERAL

O aplicativo *TattooAli* foi desenvolvido utilizando a biblioteca *React Native* com o framework *Expo*, adotando uma arquitetura baseada em camadas que promove a separação de responsabilidades entre a interface, a lógica de negócio e a comunicação com serviços externos. A estrutura organizacional do código segue o padrão de Gerenciamento de Estado via *Context API* e *Services*, garantindo que a aplicação seja escalável e de fácil manutenção.

As camadas do sistema são divididas da seguinte forma:

1. **View (Interface do Usuário):** Composta por componentes funcionais do *React Native* e telas (*Screens*). Esta camada é responsável apenas por renderizar os dados e capturar as interações do usuário, utilizando ganchos (*hooks*) para se comunicar com a lógica da aplicação.
2. **Context (Estado Global):** Camada que gerencia dados compartilhados por todo o aplicativo, como autenticação (*AuthContext*), conversas em tempo real (*ConversationsContext*) e notificações (*NotificationsContext*). Ela atua como um mediador entre a interface e os serviços de dados.
3. **Services (Serviços):** Módulos especializados em lidar com lógicas específicas, como o *chatService.js* (comunicação com *Supabase*) e serviços de *API REST* para gestão de tatuadores e sessões.



3.2. PROJETO DE DADOS: INTEGRAÇÃO COM API E PERSISTÊNCIA LOCAL

A gestão de dados do aplicativo TattooAli foi projetada para garantir a integridade das informações e uma experiência de usuário fluida, mesmo em condições de rede instável. O sistema adota o padrão de Fonte Única da Verdade (Single Source of Truth), onde o banco de dados relacional PostgreSQL (gerenciado pelo Sequelize ORM no backend) detém a versão oficial dos dados.

O fluxo de dados e a estratégia de persistência funcionam da seguinte forma:

- Sincronização com API REST:** O aplicativo mobile realiza chamadas assíncronas para os *endpoints* do backend para buscar informações em tempo real, como a lista de tatuadores disponíveis e o status das sessões de agendamento.
- Persistência e Cache Local:** Para otimizar o desempenho e permitir que o usuário acesse dados críticos offline, o aplicativo utiliza o *AsyncStorage*. O *token* JWT de autenticação e os dados essenciais do perfil do usuário são armazenados localmente, eliminando a necessidade de requisições repetitivas ao servidor a cada abertura do app.
- Realtime com Supabase:** Além da API REST, o sistema utiliza o Supabase Realtime para a camada de mensagens e notificações, garantindo que o estado das conversas seja atualizado instantaneamente no dispositivo sem sobrecarregar o banco de dados principal com consultas constantes.

Dicionário de Dados (Tabela *Lajes_Pedra*):

Nome do campo	Tipo de dados	Chave (PK/FK)	Nulo?	Descrição
user_id	UUID	PK	Não	Identificador único do usuário gerado pelo sistema.
nome	VARCHAR(100)		Não	Nome e sobrenome do cliente ou tatuador.
email	VARCHAR(150)		Não	E-mail de acesso (identificador único de login).
role	VARCHAR(20)		Sim	Papel do usuário no sistema (<i>cliente</i> ou <i>tatuador</i>).
foto	VARCHAR(255)		Sim	URL pública da imagem de perfil

				armazenada no S3.
estilo_favorito	VARCHAR(20)		Sim	Preferência estética do cliente (ex: Realismo, Fineline).
token	TEXT		Não	Token JWT persistido localmente para manter a sessão.

3.3. CONSUMO DA API E FLUXO DE NAVEGAÇÃO

instrução: Esta seção tem dois objetivos. Primeiro, referenciar a documentação oficial da API que o seu aplicativo consome. Segundo, e mais importante, apresentar um **Diagrama de Fluxo de Navegação** que mostre as telas do seu aplicativo e como o usuário transita entre elas.

Exemplo de Texto:

O aplicativo consome a API REST do sistema RochaForte, cuja documentação completa, interativa e oficial, desenvolvida com OpenAPI, pode ser acessada através do seguinte link:

- **URL da Documentação da API:**

<https://myprofile.github.io/rochaforte-api-docs/>

O fluxo de navegação do usuário dentro do aplicativo foi projetado para ser simples e intuitivo, conforme ilustrado no diagrama a seguir:

Figura 5: Diagrama de Fluxo de Navegação do Aplicativo (Legenda: O fluxo principal inicia na tela de Login. Após o sucesso, o usuário é direcionado para a Home (lista de lajes), de onde pode navegar para a tela de Detalhes de uma laje específica ou para a tela de Cadastro de uma nova laje.)

[INSERIR IMAGEM DO DIAGRAMA DE FLUXO DE NAVEGAÇÃO AQUI (SPLASH -> LOGIN -> HOME -> DETALHES)]

4. TECNOLOGIAS E FERRAMENTAS

4.1. STACK DE TECNOLOGIAS

O desenvolvimento do aplicativo *TattooAli* e de sua infraestrutura de retaguarda (backend) utilizou um ecossistema moderno focado em JavaScript/TypeScript, garantindo alta produtividade e performance. A stack tecnológica é composta por:

Frontend (Mobile App):

- **Framework Mobile:** React Native (versão 0.81) em conjunto com o Expo (SDK 54), permitindo a compilação multiplataforma nativa para Android e iOS a partir de uma base de código unificada.
- **Navegação e Estado:** Utilização do React Navigation v7 para roteamento nativo entre telas e React Context API para o gerenciamento de estado global (autenticação, conversas e notificações).
- **Persistência Local:** *AsyncStorage* para armazenamento offline seguro de tokens JWT e dados de sessão do usuário.
- **Tempo Real (BaaS):** Supabase JS SDK v2, responsável por gerenciar conexões via WebSockets (Realtime Channels) para o chat e sincronização instantânea de mensagens.

Backend (API e Banco de Dados):

- **Servidor e API:** Node.js com o framework Express 5, estruturado em arquitetura MVC para consumo RESTful pelo aplicativo mobile.
- **Banco de Dados Relacional:** PostgreSQL, gerenciado de forma segura e estruturada através do ORM Sequelize (incluindo migrations e validações).
- **Autenticação:** Supabase Auth, integrado ao banco PostgreSQL local para emissão e validação segura de JSON Web Tokens (JWT).
- **Armazenamento em Nuvem (Storage):** AWS S3, utilizado para armazenar os arquivos estáticos de mídia, como fotos de perfil e galerias de portfólio dos tatuadores.
- **Inteligência Artificial:** Integração com os modelos Google Gemini e Imagen 4 para a funcionalidade de geração de ideias de tatuagem por IA diretamente no backend.

4.2. FERRAMENTAS DE DESENVOLVIMENTO

Para apoiar o ciclo de vida do desenvolvimento, testes e implantação, a equipe fez uso das seguintes ferramentas:

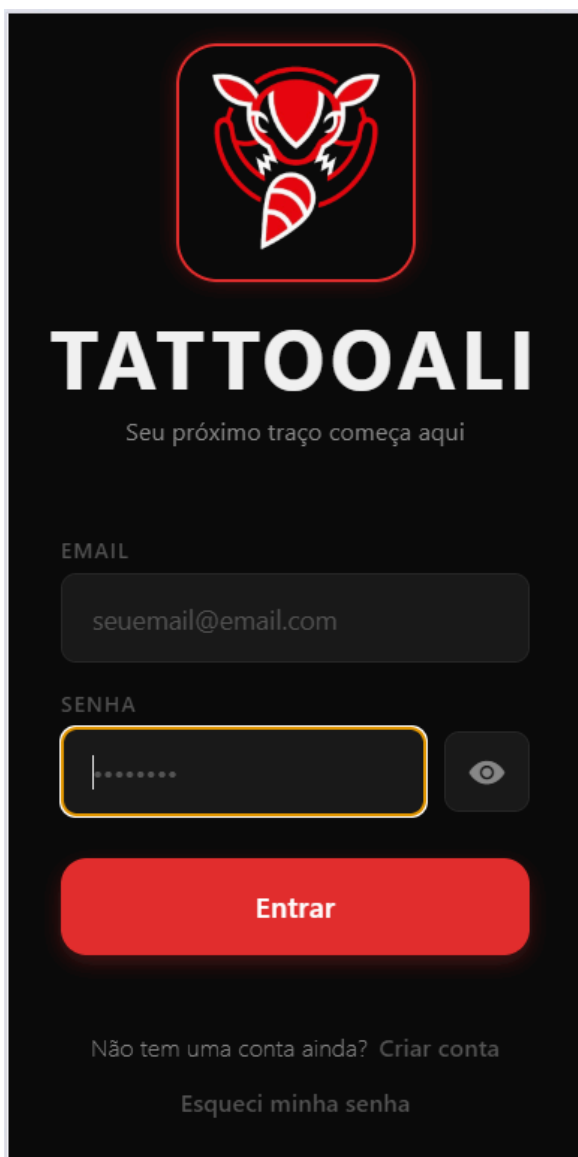
- **IDE (Ambiente de Desenvolvimento):** O Visual Studio Code foi a principal plataforma para a escrita do código do aplicativo (React Native) e da API (Node.js), graças à sua leveza e integração com terminais.
- **Controle de Versão:** Git, com o repositório hospedado no GitHub para controle de versionamento seguro do projeto.
- **Testes Automatizados:** Implementação de suítes de teste no backend utilizando Jest e Supertest para garantir a estabilidade dos endpoints e validações de schemas.
- **Documentação da API:** Swagger / OpenAPI, gerando uma interface interativa e atualizada para que o aplicativo móvel pudesse consumir os serviços de forma previsível e segura.

5. IMPLEMENTAÇÃO E RESULTADOS

5.1. TELAS DO SISTEMA

Abaixo estão apresentadas as principais interfaces gráficas do aplicativo TattooAli, demonstrando a implementação prática dos requisitos funcionais e o fluxo de interação projetado para o usuário final.

Figura 1: Tela de Entrada (Login) Legenda: A tela de login é o ponto de entrada seguro do aplicativo, garantindo o acesso às contas de clientes e tatuadores através de autenticação por e-mail e senha, com opções para recuperação de acesso e criação de nova conta.



A imagem mostra a interface de login do aplicativo TattooAli. No topo, há um ícone de uma tatuagem de tatuador dentro de um círculo vermelho. Abaixo, o nome "TATTOOALI" é exibido em letras brancas e grandes, seguido pelo slogan "Seu próximo traço começa aqui" em uma fonte menor. O formulário de login contém dois campos de entrada: "EMAIL" com o texto "seuemail@email.com" e "SENHA" com pontos para ocultar o texto. Um ícone de olho indica a opção de alternar a visibilidade da senha. Abaixo dos campos, há um botão vermelho com o texto "Entrar". Na base da tela, há dois links: "Não tem uma conta ainda? Criar conta" e "Esqueci minha senha".

Figura 2: Tela de Cadastro (Criar Conta) *Legenda: Interface dedicada ao registro de novos usuários na plataforma. O formulário solicita informações essenciais de identificação (nome, CPF, e-mail) e aplica validações de segurança para a criação da senha, integrando os dados diretamente com o serviço de autenticação.*

←

CRIAR CONTA

Junte-se à comunidade Tattoali

NOME

SOBRENOME

CPF

EMAIL

SENHA

CONFIRMAR SENHA

Figura 3: Tela de Busca e Exploração Legenda: O feed principal de busca permite que os usuários encontrem artistas rapidamente. A interface disponibiliza filtros por estilo de tatuagem (ex: 3D, Abstract, Anime) e exibe os resultados em cards resumidos contendo a nota de avaliação do profissional.

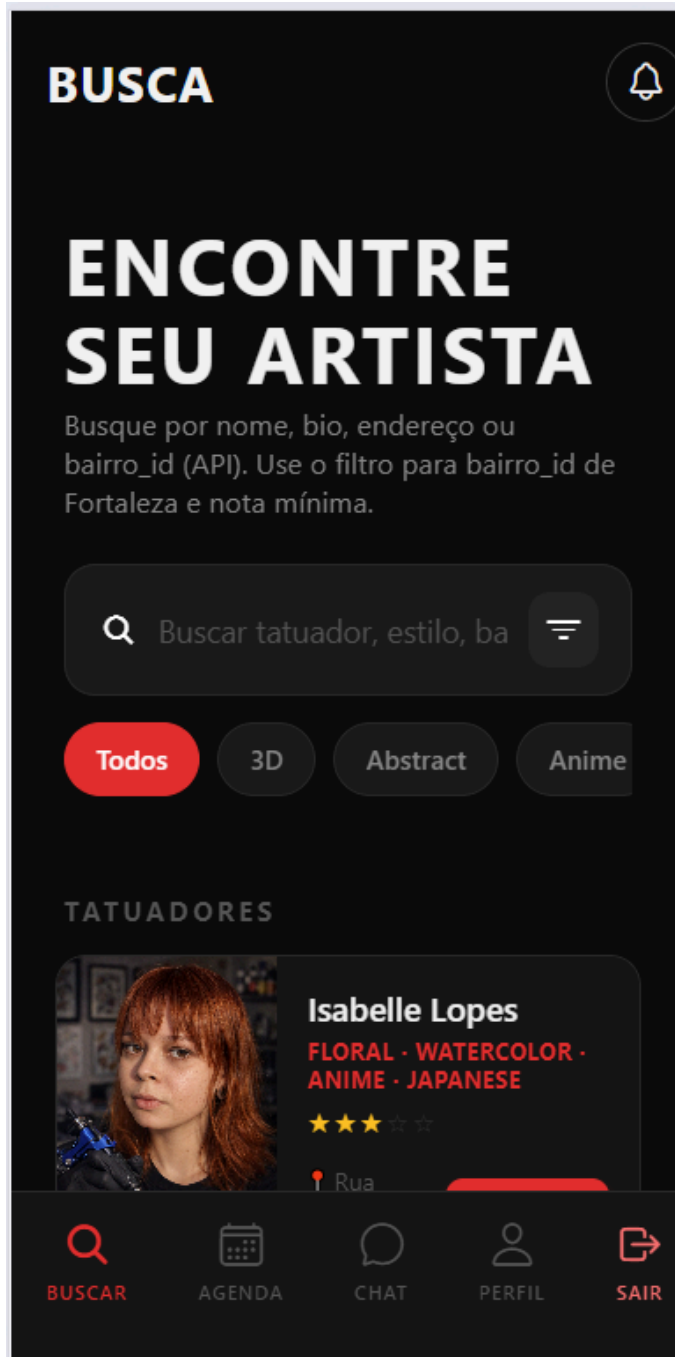


Figura 4: Perfil do Artista e Portfólio Legenda: Ao selecionar um tatuador, o cliente acessa seu perfil detalhado. Esta tela consolida as informações de localização, estilos de especialidade, botão de acesso direto ao chat e a galeria de trabalhos anteriores (portfólio) do artista.

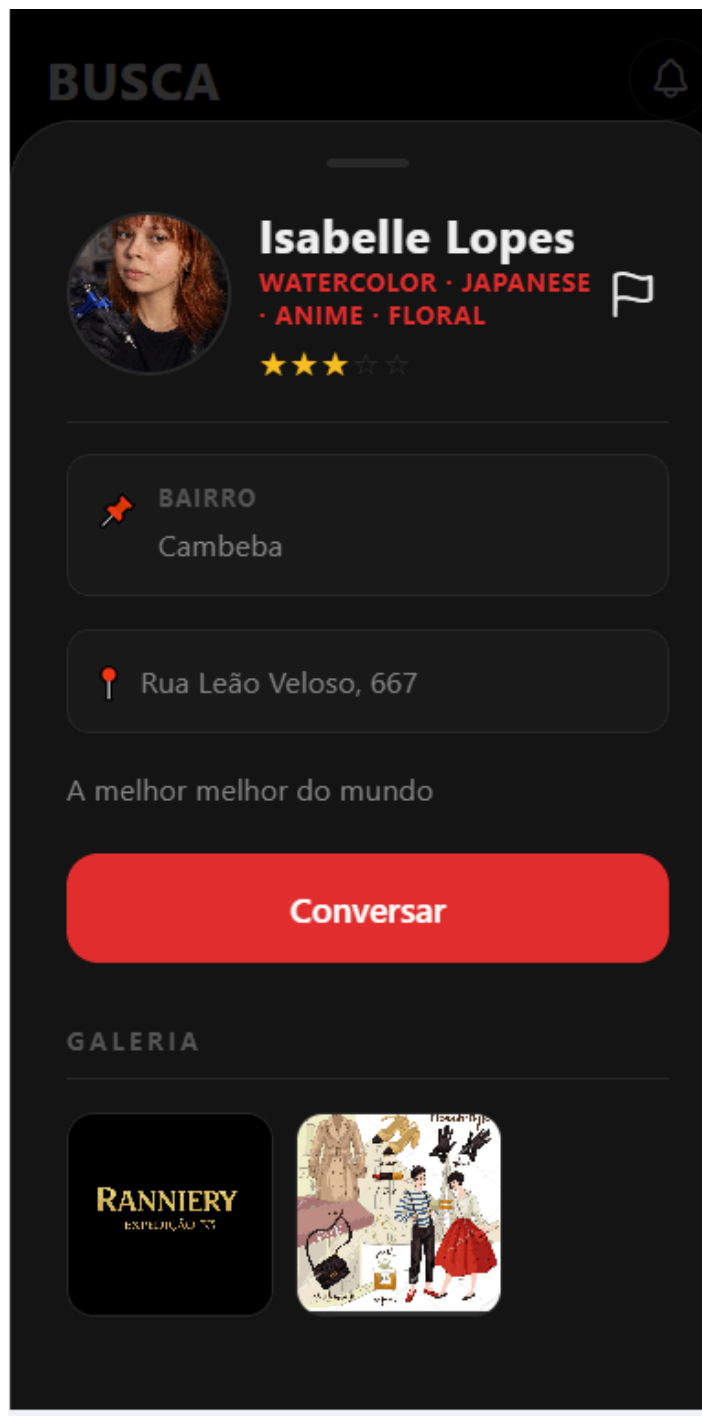


Figura 5: Gestão de Agenda e Histórico Legenda: A tela de Agenda centraliza as sessões do usuário, separando-as de forma intuitiva por abas de status: Agendadas, Concluídas e Canceladas. É a partir desta interface que o usuário pode solicitar suporte via chat ou iniciar uma avaliação.

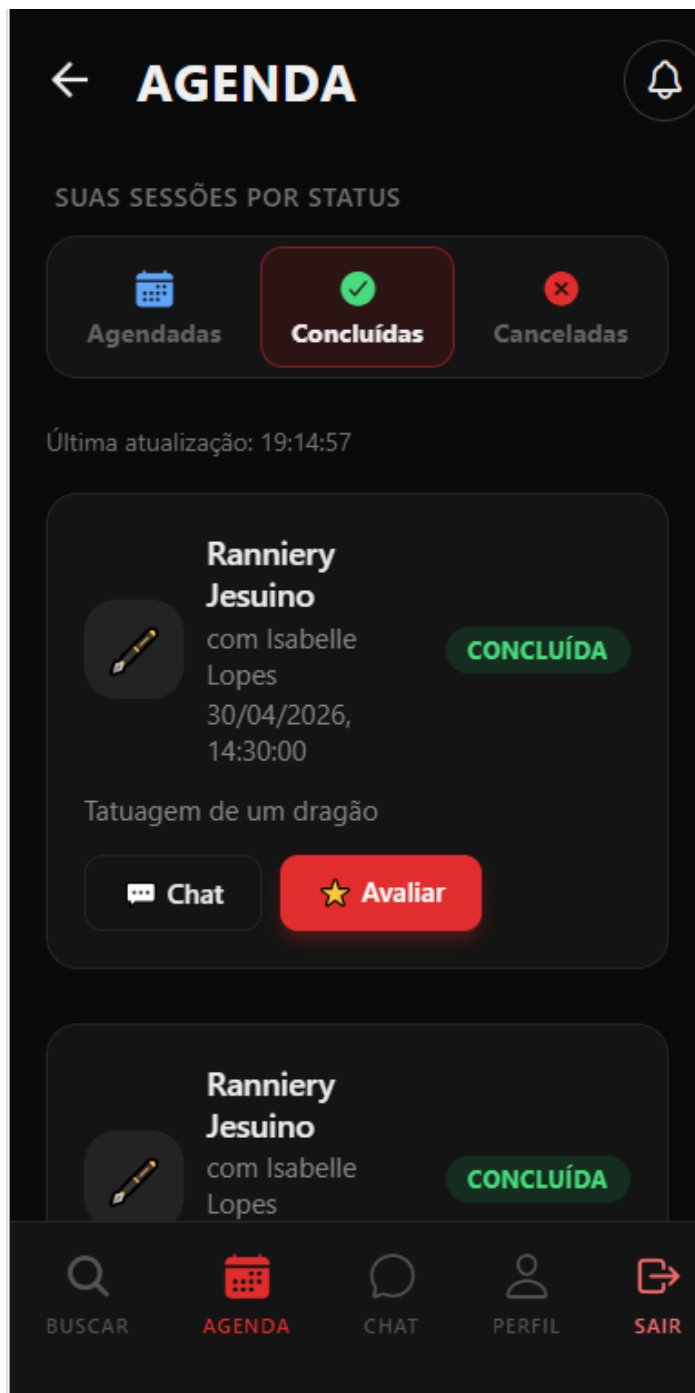


Figura 6: Modal de Avaliação de Sessão Legenda: Funcionalidade de feedback liberada após a conclusão de uma sessão. O cliente pode atribuir uma nota em formato de estrelas e deixar um comentário textual relatando sua experiência com o profissional.



← **AGENDA** 

SUAS SESSÕES POR STATUS

AVALIAR SESSÃO

Como foi sua experiência com **Isabelle Lopes**?

COMENTÁRIO (OPCIONAL)

Conte como foi sua experiência...

Enviar Avaliação

Cancelar

[illegible]

6. AMBIENTE E GUIA DE GERAÇÃO (BUILD)

Instrução: Esta seção detalha os requisitos e os passos para compilar o código-fonte e gerar o arquivo de instalação do aplicativo (o .apk).

6.1. REQUISITOS DO AMBIENTE

Instrução: Liste o software e as versões necessárias para que outra pessoa possa compilar seu projeto com sucesso.

Exemplo de Texto:

- **IDE:** Android Studio Iguana | 2023.2.1
- **Android Gradle Plugin (AGP):** 8.2.0
- **Gradle:** 8.2
- **Android SDK:** `compileSdk = 34`, `minSdk = 28`
- **JDK:** JDK 17 (embutido no Android Studio)

6.2. PROCESSO DE GERAÇÃO DE APLICATIVO

instrução: Forneça os comandos exatos para gerar uma versão de "release" do seu aplicativo a partir da linha de comando.

Exemplo de Texto:

1. Clone o repositório do projeto: `git clone https://github.com/equipe/rocha-forte-mobile.git`
2. Entre na pasta do projeto: `cd rocha-forte-mobile`
3. No Windows, execute o comando: `gradlew.bat assembleRelease`
4. No Linux ou macOS, execute o comando: `./gradlew assembleRelease`
5. Após a conclusão, o arquivo de instalação será gerado em:
`app/build/outputs/apk/release/app-release.apk`
- 1.

6.3. ACESSO À APLICAÇÃO IMPLANTADA

Instrução: Forneça um link para o download direto do arquivo .apk ou para uma plataforma de testes (como Firebase App Distribution) onde a banca possa instalar o aplicativo.

Exemplo de Texto:

- **Link para Download do APK:** [Link do Google Drive/Dropbox para o arquivo .apk]
- **Credenciais de Acesso (Perfil de Vendedor):**
 - **Usuário:** vendedor@teste.com
 - **Senha:** vendedor123

7. CONCLUSÃO

7.1. TRABALHOS FUTUROS

***Instrução:** Nenhum projeto está 100% completo. Liste aqui as melhorias e novas funcionalidades que você gostaria de implementar no futuro. Pense em como o aplicativo poderia se tornar ainda mais útil para o usuário. Isso demonstra visão de produto e consciência das limitações do trabalho atual.*

Exemplo de Texto:

Com a base sólida do aplicativo estabelecida, identificamos diversas oportunidades para evoluir e agregar ainda mais valor ao sistema RochaForte no futuro. As principais propostas são:

- **Funcionalidades Offline Avançadas:** Expandir a capacidade offline para permitir não apenas a consulta, mas também a **criação e edição de pedidos** sem conexão com a internet. Os dados seriam sincronizados automaticamente com o servidor assim que uma conexão fosse restabelecida.
- **Identificação de Lajes por QR Code:** Implementar uma funcionalidade que utilize a câmera do dispositivo para ler um QR Code fixado em cada laje de pedra. Isso permitiria ao vendedor ou gerente de estoque acessar instantaneamente os detalhes da peça, eliminando a necessidade de busca manual e agilizando o processo de inventário.
- **Notificações Push em Tempo Real:** Desenvolver um sistema de notificações push para alertar os vendedores sobre eventos importantes, como a confirmação de um pedido, a chegada de um novo lote de material ou uma alteração no status de uma laje que ele esteja monitorando.
- **Otimização para Tablets e Modo Paisagem:** Adaptar a interface do usuário para oferecer uma experiência otimizada em telas maiores, como as de tablets, que são frequentemente utilizados em balcões de vendas. Isso incluiria layouts de duas colunas e melhor aproveitamento do espaço horizontal.
- **Versão para a Plataforma iOS:** Iniciar o desenvolvimento da versão do aplicativo para iOS, a fim de atender a todos os potenciais usuários da empresa, independentemente do sistema operacional de seus dispositivos móveis.

7.2. LIÇÕES APRENDIDAS

***Instrução:** Faça uma reflexão sincera sobre a jornada de desenvolvimento do aplicativo. Quais foram os maiores desafios técnicos que a equipe enfrentou no universo mobile? Quais foram as dificuldades de integrar uma API externa? Como foi o trabalho em equipe? O que vocês fariam de diferente hoje? Esta seção valoriza o processo de aprendizado tanto quanto o resultado final.*

Exemplo de Texto:

O desenvolvimento do aplicativo RochaForte foi uma experiência de aprendizado imensa, que nos levou muito além da simples escrita de código. Nossas principais lições aprendidas podem ser divididas em três áreas:

1. **Desafios Técnicos de Arquitetura:** A maior dificuldade técnica que enfrentamos foi, sem dúvida, o gerenciamento de estado e a orquestração de operações assíncronas. Inicialmente, tínhamos dificuldade em manter a interface consistente enquanto os dados eram buscados da API e salvos no banco de dados local. Compreender e aplicar corretamente a arquitetura MVVM, utilizando `StateFlows` para expor o estado da UI a partir da `ViewModel`, foi um divisor de águas. Isso nos ensinou que uma arquitetura bem definida não é opcional, mas sim essencial para criar um aplicativo robusto e manutenível.
2. **A Importância da Persistência Local:** No início, subestimamos a complexidade de oferecer um suporte offline funcional. Implementar o padrão de Repositório como a "Fonte Única da Verdade" que abstrai a origem dos dados (API ou cache local) foi o nosso maior "Aha! Moment". Aprendemos na prática que um aplicativo moderno não apenas consome uma API, mas gerencia dados de forma inteligente para ser rápido e resiliente, melhorando drasticamente a experiência do usuário.
3. **Comunicação entre Equipes (Frontend/Backend):** A integração com a API, desenvolvida pela equipe de Web, foi um grande exercício de comunicação. Depender da documentação OpenAPI foi fundamental e nos ensinou o valor de ter um "contrato" claro entre o cliente e o servidor. Tivemos momentos em que precisávamos de um campo extra ou de um formato de dado diferente, e a negociação com a outra equipe para evoluir a API foi um aprendizado valioso sobre o desenvolvimento colaborativo no mundo real.

A principal lição que levamos deste projeto é que a qualidade de um aplicativo mobile não está apenas em sua aparência, mas em sua arquitetura resiliente e na forma como ele lida com as incertezas do mundo real, como falhas de rede.

